

03 · 4비트 가산기

VS Code 사전 시뮬레이션 → 레거시 Vivado → 보드 실험

0-15 범위의 두 수를 더한다. 결과는 carry 1비트와 합 4비트로 총 5 비트다.

1. 템플릿을 clone하고 이 회로의 workspace를 연다.
2. 예상값을 계산하고 RTL·TB·XDC를 직접 작성해 시뮬레이션한다.
3. Vivado 결과와 실제 보드 동작을 비교한다.
4. 자신의 사진·영상·레포트를 GitHub에 연결한다.

작성일 2026. 09. 10. · 이해리

실습 목차

1부 · VS Code 사전 실습

새 창·workspace·확장·코드 열기

예상값·작업 실행·로그·파형·실험 전 레포트

2부 · Vivado 실습

프로젝트·소스·top·시뮬레이션·핀·비트스트림

3부 · 결과 정리

보드·사진·영상·GitHub·실험 후 레포트

전체 목차 PDF 열기

시뮬레이션 도구 확인

Git·Python 3.10 이상·VS Code·Icarus Verilog를 설치한다. Windows PowerShell에서 한 줄씩 확인한다.

```
git --version  
python --version  
iverilog -V  
vvp -V
```

설치·PATH 변경 후 VS Code를 다시 연다. Linux·macOS·WSL은 python 대신 python3를 사용한다.

설치와 빈 템플릿 안내

v2.0.0 템플릿을 새 이름으로 clone

아래는 Windows PowerShell 명령이다. 줄 끝 문자는 다음 줄로 명령을 이어 준다.

```
git clone --branch v2.0.0 `
https://github.com/Glaysia/fpga-lab-template.git `
lab1_legacy_03_adder4
cd lab1_legacy_03_adder4
git switch -c main
code LAB1.code-workspace
```

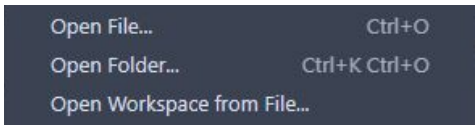
회로마다 마지막 폴더 이름을 바꾼다. 시작 파일은 주석뿐이며 코드 이미지를 보며 직접 작성한다.
고정된 수업 버전 v2.0.0

File → New Window

VS Code 상단 File → New Window. Ctrl+Shift+N도 같은 동작이다.



새 창의 File → Open Workspace from File...을 누른다.



프로젝트 하나의 workspace 열기

1. 방금 clone한 lab1_legacy_03_adder4 폴더를 연다.
2. LAB1.code-workspace를 선택하고 Open. 모든 실습의 workspace 파일명은 같다.
3. Explorer에 PROJECT 하나와 src·sim·constraints·tools 폴더가 보이는지 확인한다.
4. src/design.v·sim/tb_design.v·constraints/pins.xdc는 아직 주석뿐이다.
5. simulation.json은 실행할 RTL·TB 파일과 TB top을 지정하는 설정이다.

추천 확장 설치

1. 왼쪽 Extensions 아이콘을 누른다.
2. 검색창에 @recommended를 입력한다.
3. slang의 Install: Verilog/SystemVerilog 편집·진단.
4. VaporView의 Install: VCD 파형 확인.
5. vscode-pdf의 Install: 코드 옆에서 PDF 교안 열기.

WSL 사용자는 WSL 창에서 필요할 경우 Install in WSL을 누른다. 확장 설치와 Python·Icarus 설치의 별개다.

src에 RTL 파일 만들기

1. Explorer에서 src 왼쪽 화살표를 눌러 펼친다.
 2. design.v 우클릭 → Rename → adder_4bit.v 입력.
 3. 파일을 두 번 눌러 열고 뒤의 코드 이미지를 보며 전체 코드를 입력한다.
 4. 추가 RTL이 있으면 src 우클릭 → New File로 만든다.
 5. 전체 RTL 파일 목록은 다음 장의 src 표를 따른다.
 6. File → Save All로 모든 파일을 저장한다.
- 설계 모듈명: adder_4bit. 파일명과 module 이름을 구분한다.

sim에 테스트벤치 만들기

1. sim을 펼치고 tb_design.v 우클릭 → Rename.
2. 새 파일명: tb_adder_4bit.sv
3. 뒤의 TB 코드 이미지에 있는 선언·DUT 연결·입력 자극·예상값 검사를 직접 작성한다.
4. wave.vcd를 만드는 dumpfile·dumpvars와 종료 조건까지 작성한다.
5. TB 모듈명: tb_adder_4bit
6. File → Save All. TB를 합성용 RTL 폴더에 넣지 않는다.

작성할 RTL 파일 목록

폴더	파일명
src	adder_4bit.v

각 파일을 New File로 만들고 코드 이미지의 전체 내용을 작성한다.
파일마다 module 선언과 endmodule을 확인한다.

simulation.json을 내 파일에 맞추기

Explorer에서 simulation.json을 두 번 눌러 연다.

sources 배열: 앞의 모든 RTL 파일명 앞에 src/를 붙여 등록한다.

한 파일 예: "sources": ["src/adder_4bit.v"]

testbench: sim/tb_adder_4bit.sv

simulation_top: tb_adder_4bit

sources의 각 경로와 testbench·simulation_top 값은 큰따옴표로 감싼다.
여러 경로는 쉼표로 구분한다.

파일명·확장자·괄호·쉼표를 확인하고 Save All.

VS Code 실습은 대응 최신판을 재사용

이 PDF의 사전 실습은 03번 최신판과 같은 RTL·자기검사 TB·실행 방법이다.

실습 폴더는 이번 레거시용 이름으로 만들되, 같은 코드 이미지를 보고 직접 작성한다.

VS Code의 PASS·파형·실험 전 레포트 뒤에 이 PDF의 원본 Vivado 강의자료를 진행한다.

원본 testbench.v는 자극 순서·시간이 다를 수 있다. 동일 결과 비교에는 앞에서 작성한 자기검사 TB를 등록한다.

03번 VS Code 상세 실습 열기

입출력과 동작 규칙

입력 $a[3:0]$, $b[3:0]$ / 출력 $s[3:0]$, $cout$

0-15 범위의 두 수를 더한다. 결과는 carry 1비트와 합 4비트로 총 5 비트다.

$DIP1-4=a[3:0]$, $DIP5-8=b[3:0]$. $LED1=cout$, $LED2-5=s[3:0]$.

실행 전에 예상값 작성

입력 또는 조건	예상 결과
0+0	cout=0, s=0
1+15	cout=1, s=0
15+15	cout=1, s=14

310-320ns의 1+15에서 하위 합이 0이어도 전체 결과는 16이다.

예상값을 계산한 근거를 자신의 말로 설명한다.

RTL 구조를 설명한다

실제 배포 RTL을 VS Code에서 연 화면이다.

설계 top: adder_4bit

```
COMMON > rtl > @ adder_4bit.v
1 module adder_4bit(input wire [3:0] a,b, output wire [3:0] s, output wire cout);
2   assign {cout,s} = {1'b0,a} + {1'b0,b};
3 endmodule
```

0-15 범위의 두 수를 더한다. 결과는 carry 1비트와 합 4비트로 총 5 비트다.

포트 선언·비트 폭·출력 연결을 짚어 설명한다. 저장소 소스를 복사해 사용해도 된다.

배포 소스 확인

테스트벤치와 통과 기준

시뮬레이션 top: `tb_adder_4bit`

256개 입력 조합을 모두 검사한다. 각 자극은 10ns, 종료 시각은 2560ns다.

기대값과 실제 출력이 다르면 `$fatal`로 중단한다. 검사 횟수와 watchdog도 확인한다.

통과 문구: `LAB1_PASS adder_4bit cases=256`

원본 레거시 `testbench.v`와 별도의 자기검사 TB는 파일과 역할을 구분한다.

TB · 입력과 기대값

tb_adder_4bit.sv · 1-12행 · 실제 VS Code 화면

```
1 | timescale 1ns/1ps
2 | module tb_adder_4bit;
3 |     reg [3:0] a,b; wire [3:0] s; wire cout; adder_4bit dut(a,b,s,cout);
4 |     integer n,checked=0;
5 |     reg [4:0] expected;
6 |     initial begin
7 |         $dumpfile("wave.vcd"); $dumpvars(0,tb_adder_4bit);
8 |
9 |         for(n=0;n<256;n=n+1) begin
10 |             {a,b}=n;
11 |             expected=(n/16)+(n%16);
12 |             #10;
```

$n/16$ 은 a , $n\%16$ 은 b 다. 16×16 개 조합의 합을 5비트 기대값에 담아 carry 까지 검사한다.

DUT는 검사할 회로다. `dumpfile`·`dumpvars`는 파형 파일에 신호를 남긴다.

TB · 비교와 종료

tb_adder_4bit.sv · 13-22행 · 실제 VS Code 화면

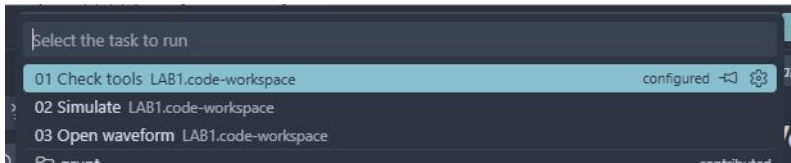
```
13     if ({cout,s} != expected)
14         $fatal(1,"FAIL adder_4bit vector=%0d expected=%h actual=%h",n,expected,{cout,s});
15     checked=checked+1;
16 end
17 if(checked!=256) $fatal(1,"Incomplete test");
18 $display("LAB1_PASS adder_4bit cases=%0d",checked);
19 $finish;
20 end
21 initial begin #100000; $fatal(1,"Watchdog"); end
22 endmodule
```

!=는 X·Z를 포함한 불일치도 잡는다. 다르면 \$fatal로 중단하고 어떤 입력에서 틀렸는지 출력한다.

checked가 전체 검사 수와 같은지 확인한 뒤 PASS와 \$finish. watchdog은 종료되지 않는 테스트를 실패시킨다.

저장 → Terminal → Run Task

File → Save All 다음 Terminal → Run Task...을 누른다.



1. 01 Check tools를 실행하고 도구 버전을 확인한다.
2. 02 Simulate를 선택하고 종료까지 기다린다.
3. 03 Open waveform으로 생성된 VCD를 연다.

작업이 없으면 올바른 .code-workspace를 열었는지 확인한다.

실행 로그 확인

터미널과 `build/sim/run-.../simulation.log`에서 이번 실행을 확인한다.

`LAB1_PASS adder_4bit cases=256`

정상 종료 시각: 2560ns. PASS 문구와 검사 수를 함께 확인한다.

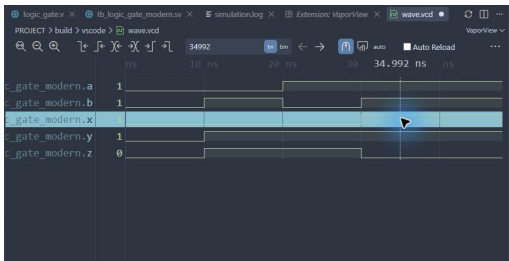
실패하면 이번 실행 폴더의 `compile.log`와 `simulation.log`에서 첫 오류를 찾는다.

코드 저장 → 02 Simulate 재실행 → 새 로그 확인 순서로 복귀한다.
프로젝트 검증 안내

VCD를 파형으로 열기

1. 03 Open waveform 또는 build/sim/wave.vcd를 연다.
2. 텍스트로 열리면 탭 우클릭 → Reopen Editor With... → VaporView.
3. Netlist View에서 tb_adder_4bit을 펼친다.
4. 신호 a[3:0], b[3:0], s[3:0], cout를 각각 두 번 눌러 추가한다.
5. Zoom to Fit를 누르고 Time Units를 ns로 맞춘다.
6. 전환 경계 대신 구간 중간에 커서를 두고 값을 읽는다.

파형 화면의 읽는 순서



위는 공통 파형 조작 예다. 이번 회로에서는 앞 장의 신호 이름을 선택한다.

310–320ns의 1+15에서 하위 합이 0이어도 전체 결과는 16이다.

신호 이름 → 시간 구간 → 커서 값 → 예상 결과 순서로 비교한다.

실제 VCD의 구간 중간값

시간(ns)	a[3:0]	b[3:0]	s[3:0]	cout
5	0000	0000	0000	0
15	0000	0001	0001	0
35	0000	0011	0011	0
1285	1000	0000	1000	0
2555	1111	1111	1110	1

이 표는 실행된 VCD에서 읽은 값이다. 다중 비트는 이진수다.

표의 시간으로 파형 커서를 옮겨 직접 대조하고, 추가 경계 조건도 확인한다.

constraints에 XDC 직접 작성

1. constraints를 펼치고 pins.xdc 우클릭 → Rename.
2. 파일명: adder_4bit.xdc
3. 핀 표와 코드 이미지를 보며 PACKAGE_PIN·IOSTANDARD·get_ports를 입력한다.
4. get_ports의 이름과 비트 번호를 자신이 작성한 RTL의 포트와 대조한다.
5. File → Save All. Vivado 또는 CLI 구현에는 이 파일을 직접 가져간다.

XDC는 Icarus 기능 시뮬레이션에서 사용하지 않는다. 파형 통과만으로 핀 배치까지 검증됐다고 판단하지 않는다.

XDC 전체 코드 · 1-14행

constraints/adder_4bit.xdc · 실제 VS Code 화면을 보며 직접 작성한다.

```
1  # Derived from vendor archive; see legacy provenance.
2  set_property PACKAGE_PIN L4 [get_ports cout]
3  set_property PACKAGE_PIN Y1 [get_ports {a[3]}]
4  set_property PACKAGE_PIN W3 [get_ports {a[2]}]
5  set_property PACKAGE_PIN U2 [get_ports {a[1]}]
6  set_property PACKAGE_PIN T1 [get_ports {a[0]}]
7  set_property PACKAGE_PIN W4 [get_ports {b[3]}]
8  set_property PACKAGE_PIN W1 [get_ports {b[2]}]
9  set_property PACKAGE_PIN V4 [get_ports {b[1]}]
10 set_property PACKAGE_PIN U4 [get_ports {b[0]}]
11 set_property PACKAGE_PIN M4 [get_ports {s[3]}]
12 set_property PACKAGE_PIN N7 [get_ports {s[1]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports cout]
14 set_property IOSTANDARD LVCMOS33 [get_ports {a[3]}]
```

핀 이름·포트 이름·대괄호·중괄호를 확인하고 저장한다.

XDC 전체 코드 · 15-28행

constraints/adder_4bit.xdc · 실제 VS Code 화면을 보며 직접 작성한다.

```
15 set_property IOSTANDARD LVCMOS33 [get_ports {a[2]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {a[1]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {a[0]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {b[3]}]
19 set_property IOSTANDARD LVCMOS33 [get_ports {b[2]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {b[1]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports {b[0]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {s[3]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {s[2]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {s[1]}]
25 set_property IOSTANDARD LVCMOS33 [get_ports {s[0]}]
26
27 set_property PACKAGE_PIN M2 [get_ports {s[2]}]
28 set_property PACKAGE_PIN M7 [get_ports {s[0]}]
```

핀 이름·포트 이름·대괄호·중괄호를 확인하고 저장한다.

코드 수정 실험 · 실패와 복구

1. 정상 PASS 로그를 보관한다. Explorer → src → adder_4bit.v을 연다.
2. 덧셈 연산자를 뺄셈으로 바꾼다. 테스트벤치의 기대값은 그대로 둔다.
3. File → Save All → Terminal → Run Task... → 02 Simulate.
4. $a=0$, $b=1$ 의 정상 결과는 $carry=0$, $sum=10$ 이다. 변경 후 $carry=1$, $sum=15$ 다.
5. 이번 실행의 실패 로그에서 입력·기대값·실제값을 읽는다. 실행 폴더를 기록한다.
6. 바꾼 부분을 원래대로 복구하고 Save All → 02 Simulate. 전체 PASS와 새 파형을 확인한다.

사전 레포트에 변경 이유·실패 원인·복구 결과를 비교한다. 복구 PASS 뒤에 구현으로 진행한다.

제작 검증: 정상 → 실패 검출 → 복구

실험 전 레포트

1. 목적·포트·비트 폭·예상표와 동작 원리를 설명한다.
 2. 소스와 TB 링크, 코드 커밋, 도구 버전을 남긴다.
 3. 입력 조합·기대값·자동 비교·종료 조건을 설명한다.
 4. 자신의 PASS 로그와 파형 원본·캡처를 첨부한다.
 5. 시간 구간별 값을 해석하고 예상값과 비교한다.
 6. 오류·수정·재실행, 보드에서 확인할 입력과 출력을 적는다.
- 교재 캡처를 자신의 실행 결과로 제출하지 않는다.

원문 Vivado 실습으로 이동

이후 원문은 원본의 사진·문구·순서를 유지한다.

배포 프로젝트는 Vivado 2020.1 원본 XPR 형식을 기반으로 상대경로를 정리했다. 구버전 실행 검증은 별도로 확인한다.

수업에서는 원문의 합성·구현에 앞서 자기검사 TB로 Behavioral Simulation을 먼저 실행한다.

원본 testbench.v와 검증용 TB의 입력 순서·실행 시간은 서로 다를 수 있다.

[실습 13] 4비트 덧셈 회로 설계

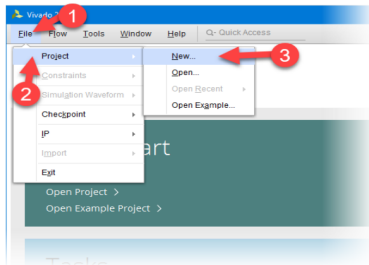
서울시립대학교 전자전기컴퓨터공학부

설계 기획

- 4비트의 입력 $a[3..0]$ 와 $b[3..0]$ 을 더함
- $s[3..0]$ 에 출력, Carry가 발생하면 cout에 출력

로직 설계 및 합성

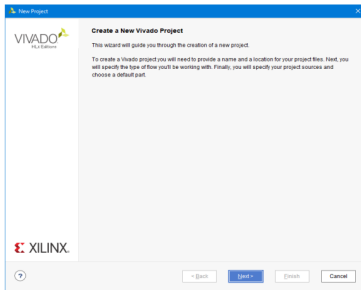
- (1) 프로젝트 선언
 - Vivado 실행
 - File > Project > New



서울시립대학교 전자전기컴퓨터공 학부

로직 설계 및 합성

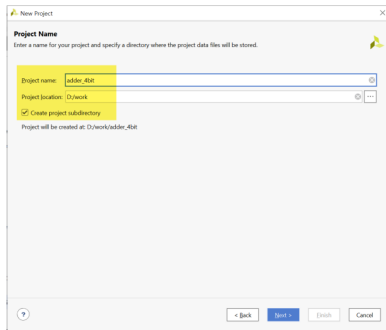
- (1) 프로젝트 선언
 - Next



서울시립대학교 전자전기컴퓨터공 학부

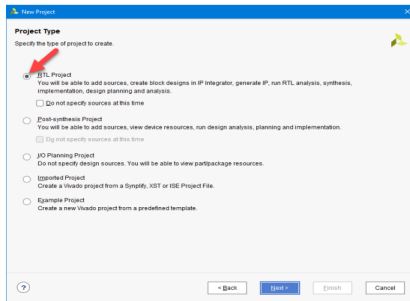
로직 설계 및 합성

- (1) 프로젝트 선언
 - Project Name과 Project Location 설정
 - Project Name : adder_4bit
 - Project location : d:/work
 - Create project subdirectory : 체크



로직 설계 및 합성

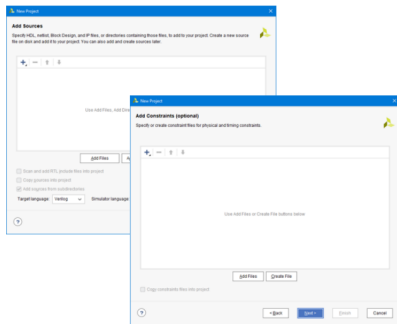
- (1) 프로젝트 선언
 - Project Type 설정
 - RTL Project



서울시립대학교 전자전기컴퓨터공학부

로직 설계 및 합성

- (1) 프로젝트 선언
 - Add source와 Add Constraints 설정
 - 설계된 로직 파일과 설정된 핀 정보 파일이 없기 때문에 Next 버튼 누름.



서울시립대학교 전자전기컴퓨터공학부

로직 설계 및 합성

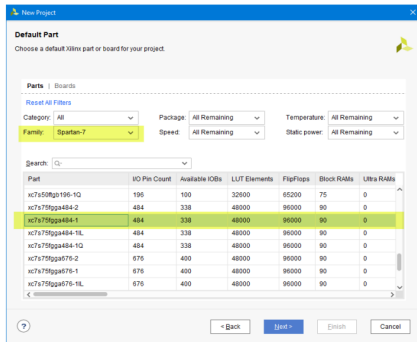
■ (1) 프로젝트 선언

■ Device 설정

- Family : Spartan-7
- Part : xc7s75fgga484-1

■ 설정된 Summary 확인.

■ Finish를 눌러 프로젝트 선언 완료



서울시립대학교 전자전기컴퓨터공학부

로직 설계 및 합성

■ (2) 로직 설계

- VIVADO 프로그램
- Text Editor > New File.. 선택
- adder_4bit.v 파일로 저장

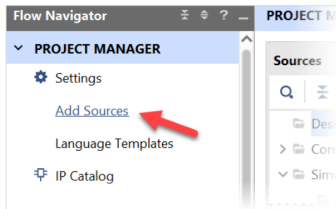
```
2 module adder_4bit( a, b, s, cout);  
3 :  
4 : input [3:0] a, b;  
5 :  
6 : output [3:0] s;  
7 : output cout;  
8 :  
9 : reg [3:0] s;  
10 : reg cout;  
11 :  
12 :  
13 : always @(a or b)  
14 : begin  
15 :     s = a + b;  
16 :  
17 :     if ((a+b) > 5'b01111)    cout = 1'b1;  
18 :     else                    cout = 1'b0;  
19 : end  
20 :  
21 : endmodule
```

서울시립대학교 전자전기컴퓨터공학부

로직 설계 및 합성

■ (2) 로직 설계

- PROJECT MANAGER > Add Sources 선택



서울시립대학교 전자전기컴퓨터공학부

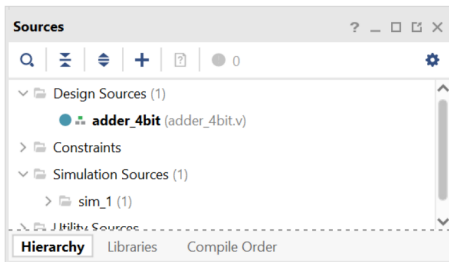
로직 설계 및 합성

■ (2) 로직 설계

- Add Sources 창에서

Add or Create design sources 선택

- adder_4bit.v 파일 추가



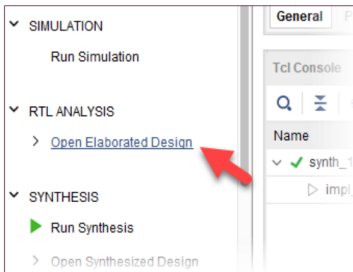
서울시립대학교 전자전기컴퓨터공학부

로직 설계 및 합성

■ (3) Assignment

- PROJECT MANAGER의 RTL ANALYSIS 탭의

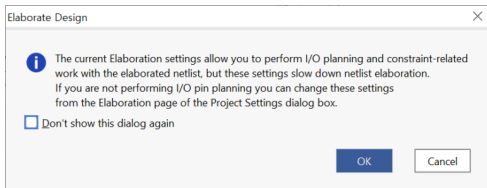
Open Elaborated Design 부분을 마우스로 클릭



로직 설계 및 합성

■ (3) Assignment

- 메시지 창이 나타난다.
- OK 버튼을 누르면 Elaboration 과정이 진행됨.
 - 문법에 오류부분도 같이 체크하여
오류가 있다면 에러 메시지가 출력됨.

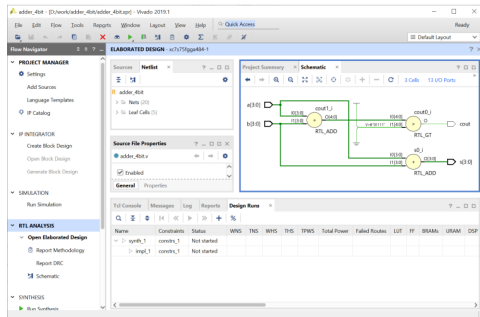


로직 설계 및 합성

■ (3) Assignment

- 설계된 Verilog HDL 구문이

Schematic으로 표현되는 것을 확인



서울시립대학교 전자전기컴퓨터공학부

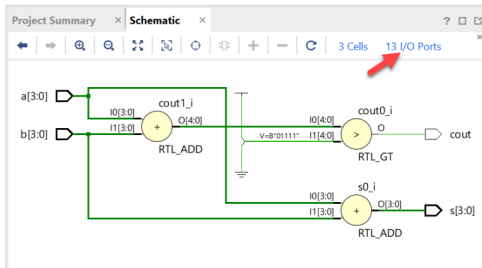
로직 설계 및 합성

■ (3) Assignment

■ Schematic 화면

오른 쪽 윗 부분의

I/O Ports 선택



로직 설계 및 합성

■ (3) Assignment

- I/O Std를 “LVCMOSS33” 으로 변경
- 입/출력 포트의 Voltage Level을 3.3V로 설정하는 것.

포트 이름	핀 번호		포트 이름	핀 번호	
a[3]	Y1	DIPSW_1	cout	L4	LED1
a[2]	W3	DIPSW_2	s[3]	M4	LED2
a[1]	U2	DIPSW_3	s[2]	M2	LED3
a[0]	T1	DIPSW_4	s[1]	N7	LED4
b[3]	W4	DIPSW_5	s[0]	M7	LED5
b[2]	W1	DIPSW_6			
b[1]	V4	DIPSW_7			
b[0]	U4	DIPSW_8			

로직 설계 및 합성

■ (3) Assignment

- Package Pin 부분에 핀 설정
- I/O Std를 “LVCMOSS33” 으로 변경
 - 입/출력 포트의 Voltage Level을 3.3V로 설정하는 것.

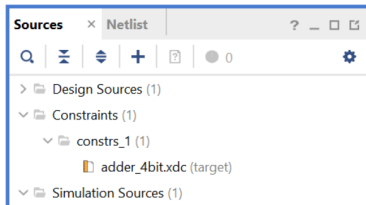
Name	Direction	Interface	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco
cout	OUT			L4	✓	35	LVCMOS33*	3.300
a[3]	IN			Y1	✓	34	LVCMOS33*	3.300
a[2]	IN			W3	✓	34	LVCMOS33*	3.300
a[1]	IN			U2	✓	34	LVCMOS33*	3.300
a[0]	IN			T1	✓	34	LVCMOS33*	3.300
b[3]	IN			W4	✓	34	LVCMOS33*	3.300
b[2]	IN			W1	✓	34	LVCMOS33*	3.300
b[1]	IN			V4	✓	34	LVCMOS33*	3.300
b[0]	IN			U4	✓	34	LVCMOS33*	3.300
s[3]	OUT			M4	✓	35	LVCMOS33*	3.300
s[2]	OUT			M2	✓	35	LVCMOS33*	3.300
s[1]	OUT			N7	✓	35	LVCMOS33*	3.300
s[0]	OUT			M7	✓	35	LVCMOS33*	3.300

서울시립대학교 전자전기컴퓨터공 학부

로직 설계 및 합성

■ (3) Assignment

- 핀 설정 저장
- 파일명은 adder_4bit로 설정



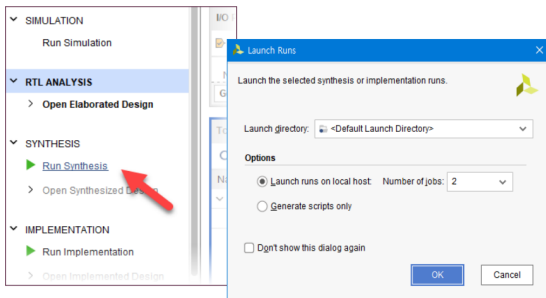
로직 설계 및 합성

■ (4) Compile

■ SYNTHESIS

> Run Synthesis

- 설계된 Verilog HDL 코드를 합성하여 최적화 시키는 과정



서울시립대학교 전자전기컴퓨터공학부

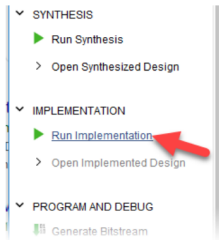
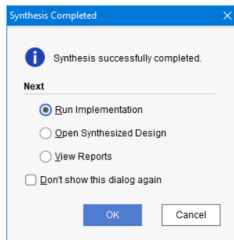
로직 설계 및 합성

■ (4) Compile

■ Synthesis 후 IMPLEMENTATION 진행

■ Implementation

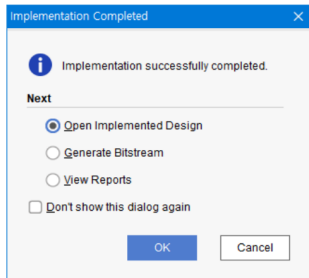
- 합성한 결과를 이용하여
실제 FPGA의 Logic Cell 단위로 바꾸어
FPGA내에 위치시키고 연결하는
Place & Route를 하는 과정



로직 설계 및 합성

■ (4) Compile

- Implementation 이 끝난 화면
- Open Implemented Design 선택 후 OK
- 결과 확인



로직 설계 및 합성

■ (5) Simulation

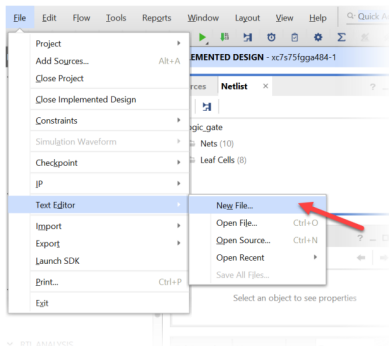
- 소프트웨어적으로 검증하는 과정
- 테스트 벤치 파일이 필요함.
 - 시뮬레이션에 대한 입력 조건을 설정한 파일

로직 설계 및 합성

■ (5) Simulation

- File > Text Editor > New File

선택



서울시립대학교 전자전기컴퓨터공 학부

로직 설계 및 합성

■ (5) Simulation

- testbench.v 파일로 저장

```
1  `timescale 10ns / 100ps
2
3  module testbench();
4      // input
5      reg [3:0] a, b;
6      // output
7      wire cout;
8      wire [3:0] s;
9      // Instantiate the U1
10     adder_4bit u1(a, b, s, cout);
11     // Specify input stimulus
12     initial begin
13         a = 4'b0000; b = 4'b0000;
14         #10 a = 4'b1010; b = 4'b0111;
15         #10 a = 4'b0110; b = 4'b0101;
16         #10 a = 4'b0101; b = 4'b0101;
17         #10 a = 4'b0010; b = 4'b1001;
18     end
19
20 endmodule
```

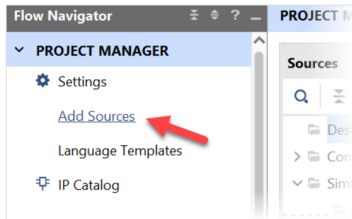
서울시립대학교 전자전기컴퓨터공학부

로직 설계 및 합성

■ (5) Simulation

■ PROJECT MANAGER

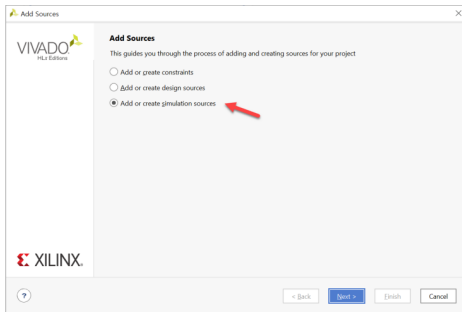
Add Source 선택



로직 설계 및 합성

■ (5) Simulation

- [Add or create simulation source] 선택
- Next
- Add Sources 창에서 [Add Files]을 선택
- testbench.v 파일 추가



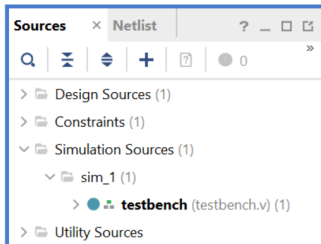
서울시립대학교 전자전기컴퓨터공 학부

로직 설계 및 합성

■ (5) Simulation

■ Simulation Sources

파일 추가 확인

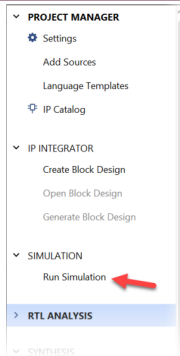


로직 설계 및 합성

■ (5) Simulation

■ SIMULATION > Run Simulation

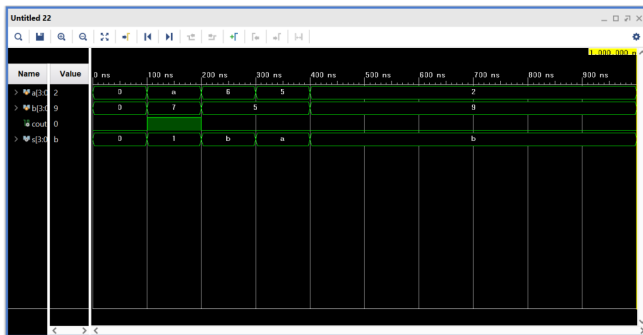
■ Run Behavioral Simulation 실행



로직 설계 및 합성

■ (5) Simulation

■ 결과 확인



서울시립대학교 전자전기컴퓨터공 학부

로직 설계 및 합성

■ (6) Programming

■ Project Manager 창

PROGRAM AND DEBUG

> Generate Bitstream

> RTL ANALYSIS

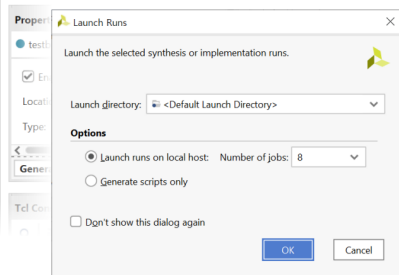
> SYNTHESIS

> IMPLEMENTATION

▼ PROGRAM AND DEBUG

Generate Bitstream

> [Open Hardware Manager](#)



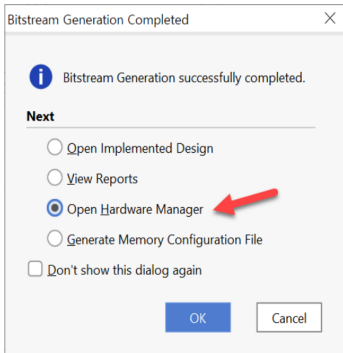
서울시립대학교 전자전기컴퓨터공 학부

로직 설계 및 합성

■ (6) Programming

■ 프로그래밍 파일 생성 완료 후

■ Open Hardware Manager



로직 설계 및 합성

- (6) Programming
 - 하드웨어 준비



서울시립대학교 전자전기컴퓨터공학부

로직 설계 및 합성

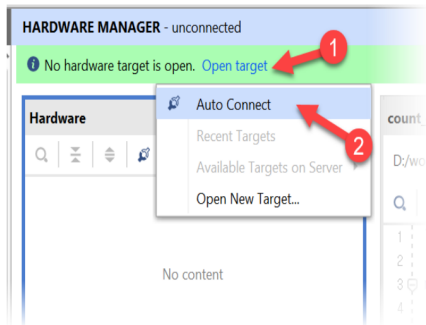
■ (6) Programming

- 장비의 전원이 꺼져 있는지 확인
- Xilinx Platform Cable에 USB Cable의 한 쪽 끝을 연결
- 반대쪽 끝은 PC의 USB 포트에 연결
- Xilinx Platform Cable에 연결된 2x14핀의 플랫 케이블은 장비의 Xilinx 모듈의 JTAG 포트에 연결
- 장비의 전원을 켜다.

로직 설계 및 합성

■ (6) Programming

- Open Target > Auto Connect

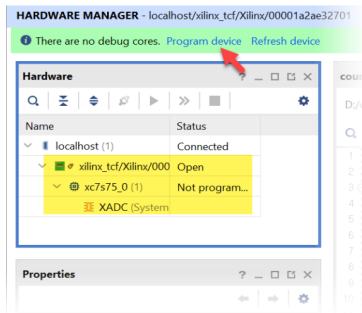


서울시립대학교 전자전기컴퓨터공학부

로직 설계 및 합성

■ (6) Programming

- Program Device 선택



서울시립대학교 전자전기컴퓨터공 학부

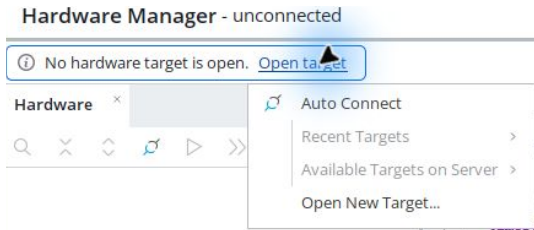
로직 설계 및 합성

■ (6) Programming

- Bitstream File 선택
- <Project Folder> ₩ adder_4bit.runs ₩ impl_1₩ adder_4bit.bit 파일 선택
- Program 버튼 클릭

■ (7) 동작 확인

실제 보드 연결



Open Hardware Manager → Open target → Auto Connect.

DIP1-4=a[3:0], DIP5-8=b[3:0]. LED1=cout, LED2-5=s[3:0].

장치가 없으면 보드 전원·USB JTAG·드라이버·VM USB 연결을 점검한다.

Program Device와 동작 확인

1. 연결된 장치 이름이 xc7s75인지 확인한다.
2. 장치 우클릭 → Program Device.
3. 이번 프로젝트의 adder_4bit.bit를 선택하고 Program.
4. 기록 완료를 확인한 뒤 입력을 바꾸고 출력과 예상값을 비교한다.
5. 기록 성공 화면과 실제 보드 동작은 각각 증빙한다.

제작 환경의 실제 보드 기록·촬영 검증 여부는 검증표를 따른다. 파일 생성만으로 보드 동작 성공을 선언하지 않는다.

사진과 시연 영상 촬영

DIP1-4=a[3:0], DIP5-8=b[3:0]. LED1=cout, LED2-5=s[3:0].

사진에는 보드 연결과 입력·출력 위치가 함께 보이게 한다.

영상에는 입력을 조작하는 과정과 그에 따른 출력 변화를 담는다.

예상표의 정상·경계 조건을 직접 보여주고 회로 번호를 파일명에 적는다.

통합 영상이라면 각 회로의 타임스탬프를 레포트에서 연결한다.

GitHub 결과 정리

1. reports/pre와 reports/post에 실험 전·후 레포트를 둔다.
 2. evidence/vscode와 evidence/vivado에 로그·파형·캡처를 구분한다.
 3. evidence/board/photos와 videos에 직접 촬영한 자료를 둔다.
 4. 큰 영상·bit는 배포 위치를 정하고 README와 레포트에서 연결한다.
 5. 웹에서 사진 표시와 영상 재생 또는 다운로드가 되는지 확인한다.
- 레포트 양식·예시·GitHub 안내

실험 후 레포트

1. Vivado 버전·part·top·핀 제약·코드 커밋을 기록한다.
2. VS Code와 Vivado의 입력·출력·시간·검사 수를 비교한다.
3. 합성·구현·bit 경로와 경고·수정 사항을 설명한다.
4. 장치 기록 화면·사진·영상과 해당 조건을 연결한다.
5. 예상값·두 시뮬레이션·실측의 일치 또는 차이 원인을 해석한다.
6. 미수행 항목은 미완료로 남기고 후속 확인을 적는다.

전체 목차 PDF 프로젝트로 돌아가기